inplace_vector

> A dynamically-resizable vector with fixed capacity and embedded storage

Document number: P0843R14.
Date: 2024-06-26.
Authors: Gonzalo Brito Gadeschi, Timur Doumler <papers _at_ timur.audio>, Nevin Liber, David Sankel <dsankel _at_ adobe.com (http://adobe.com)>.
Reply to: Gonzalo Brito Gadeschi <gonzalob _at_ nvidia.com (http://nvidia.com)>.
Audience: LWG.

Table of Contents

    - Name
- Technical specification
    - [library.requirements.organization.headers]
    - [iterator.range] Range access
    - [container.alloc.reqmts]
    - [allocator.requirements.general]
    - [containers.general]
    - [container.reqmts] General container requirements
    - [containers.sequences.general]
    - [container.requirements.sequence.reqmts]
    - [containers.sequences.inplace.vector.syn] Header <inplace_vector> synopsis
    - [containers.sequences.inplace.vector] Class template inplace_vector
        - [containers.sequences.inplace.vector.overview] Overview
        - [containers.sequences.inplace.vector.cons] Constructors
        - [containers.sequences.inplace.vector.capacity] Size and capacity
        - [containers.sequences.inplace.vector.data] Data
        - [containers.sequences.inplace.vector.modifiers] Modifiers
        - [containers.sequences.inplace.vector.erasure] Erasure
    - [version.syn]
    - [diff.cpp23.library] Compatibility
- Acknowledgments

Changelog

- **Revision 13**
    - Change the conditionally-constexpr wording and enable only for `is_trivial_t<T>` is true.
- **Revision 12**
    - Conditionally `constexpr` as per older revision.
    - Re-introduce `constexpr` discussion (dropped in R8).
- **Revision 11**
    - Container `erase` / `erase_if` non-member functions made freestanding.
    - Require `is_nothrow_move_constructible_v<T>` for `operator=(inplace_vector&& other)`.
    - Remove `[[nodiscard]]` from `empty`.
- **Revision 10**
    - Extend `constexpr` from "trivial" types to "literal" types.

    - Remove `unchecked_append_range` : adds very little value (one branch amortized over all inserted elements).
    - Update remarks of mutating elements to specify that if an exception occurs while inserting elements, the succesfully inserted elements are kept.
    - Discussion of execption safety guarantees for mutating operations and outcome from LEWG discussion.
    - Should not be allocator away.
    - Should throw `bad_alloc` on exceeding capacity.
    - Should be in a separate header.
    - Added fallible `append_range` APIs.
    - Move iterator erase methods from [vector.erasure] to [vector.modifiers].
    - Updated some EDITORIAL notes.
    - Fixed typo in [vector.modifiers], the `insert_range` method was incorrectly named `insert` .
    - Add accidentally missing `append_range` to [vector.modifiers].
    - Removed unnecessary *Complexity* clauses from `resize` methods.
- **Revision 9** Varna 2023
    - All preconditions on `sz < capacity` are now a "Throws `bad_alloc` " with the exception of the " `unchecked_` " family of functions.
    - The " `try_` " family of insertion functions do not consume the input rvalue references if the container is full.
    - Container move / copy constructor are trivial if `T` is trivial move / copy constructible.
    - Swap member function is now noexcept if `N == 0` or value type has nothrow move constructors.
    - Made complexity of resize linear.
    - Fixed out-of-bounds math in wording (less than equal to vs less).
    - Fixed constraints on all the " `emplace` family" of functions.
    - Fixed constraints of unary constructor taking a size to require default insertability instead of copy insertability.
    - Fixed missing angle brackets on `<inplace_vector>` header and listed headers alphabetically.
    - Cleanup: removed duplicates of preconditions that are covered in the sequence container requirements.
    - Cleanup: removed unnecessary specification of member swap and specialized algorithms.

    - Styling: use `class` instead of `typename` in template heads, replace `value_type` with `T` in wording, `bad_alloc` in code font, etc.
- **Revision 8** Varna 2023
    - Added LEWG poll showing consensus for `<inplace_vector>` header.
    - Add feature test macro
    - Add `try_push_back` and `unchecked_push_back` to wording.
    - Add `at` to `inplace_vector` class synopsis.
    - Add range construction and assignment.
    - Add missing `reserve` method that throws `bad_alloc` if `capacity()` is exceeded.
    - Add missing `shrink_to_fit` method that has no effects.
    - Add missing `insert_range` .
    - Add wording for move constructor semantics (trivial if `T` is trivial).
    - Add wording for destructor semantics (trivial if `T` is trivial).
    - Remove deduction guidelines since cannot deduce `capacity()` meaningfully.
    - Add to containers.sequences.general.
    - Add to sequence containers table.
    - Add to iterator.range.
    - Add to diff.cpp03.library.
    - Add poll result confirming unchecked_push_back.
    - Add erasure.
    - Add poll result confirming the overall design.
    - Review synopsis/wording for other missing functions.
    - Update `operator==` to `operator<=>` using hidden friends for them.
    - Made `<inplace_vector>` not freestanding (this will be handled in a separate paper).
- **Revision 7** Varna 2023
    - Rename `static_vector` to `inplace_vector` throughout.
    - Update `try_push_back` APIs to return `T*` with rationale.
    - Update `push_back` to throw `std::bad_alloc` with rationale .
    - Trivially-copyable if `value_type` is trivially-copyable.
    - Request LEWG poll regarding ``<vector> `or` <inplace_vector>` header.
    - Make `push_back` return a reference
- **Revision 6**: for Varna 2023 following Kona's 2022 guidance
    - Updated push_back semantics to follow std::vector (note about exception to throw).
    - Added `try_push_back` returning an `optional`
    - Added `push_back_unchecked` : excedding capacity exhibits undefined behavior.

    - Added note about naming.
- **Revision 5**:
    - Update contact wording and contact data.
    - Removed naming discussion, since it was resolved (last available in P0843r4 (https://wg21.link.p0843r4)).
    - Removed future extensions discussion (last available in P0843r4 (https://wg21.link.p0843r4)).
    - Addressed LEWG feedback regarding move-semantics and exception-safety.
- **Revision 4**:
    - LEWG suggested that push_back should be UB when the capacity is exceeded
    - LEWG suggested that this should be a free-standing header
- **Revision 3**:
    - Include LWG design questions for LEWG.
    - Incorporates LWG feedback.
- **Revision 2**
    - Replace the placeholder name `fixed_capacity_vector` with `static_vector`
    - Remove at checked element access member function.
    - Add changelog section.
- **Revision 1**
    - Minor style changes and bugfixes.

Introduction

This paper proposes `inplace_vector`, a dynamically-resizable array with capacity fixed at compile time and contiguous inplace storage, that is, the array elements are stored within the vector object itself. Its API closely resembles `std::vector<T, A>`, making it easy to teach and learn, and the *inplace storage* guarantee makes it useful in environments in which dynamic memory allocations are undesired.

This container is widely-used in the standard practice of C++, with prior art in, e.g., `boost::static_vector<T, Capacity>` [1] (http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html) or the EASTL [2] (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71), and therefore we believe it will be very useful to expose it as part of the C++ standard library, which will enable it to be used as a vocabulary type.

Motivation and Scope

The `inplace_vector` container is useful when:

- memory allocation is not possible, e.g., embedded environments without a free store, where only automatic storage and static memory are available;
- memory allocation imposes an unacceptable performance penalty, e.g., in terms of latency;
- allocation of objects with complex lifetimes in the *static*-memory segment is required;
- the storage location of the `inplace_vector` elements is required to be within the `inplace_vector` object itself, e.g., for serialization purposes (e.g. via `memcpy`);
- `std::array` is not an option, e.g., if non-default constructible objects must be stored; or
- a dynamically-resizable array is needed during constant evaluation.

Existing practice

Three widely used implementations of `inplace_vector` are available: Boost.Container [1] (http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html), EASTL [2] (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71), and Folly [3] (https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md). `Boost.Container` implements `inplace_vector` as a standalone type with its own guarantees. EASTL (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71) and Folly (https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md) implement it via an extra template parameter in their `small_vector` types.

Custom allocators like Howard Hinnant's `stack_alloc` [4] (https://howardhinnant.github.io/stack_alloc.html) emulate `inplace_vector` on top of `std::vector`, but as discussed in the next sections, this emulation is not great.

Other prior art includes the following.

- P0494R0: `contiguous_container` proposal [5] (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0494r0.pdf): proposes a `Storage` concept.
- P0597R0: `std::constexpr_vector<T>` [6] (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0597r0.html): proposes a vector that can only be used in constexpr contexts.

A reference implementation of this proposal is available here (godbolt) (https://godbolt.org/z/5P78aG5xE).

Design

The design described below was approved at LEWG Varna '23 ():

- **POLL:** The provided signatures and semantics that D08437R7 provides for push_back, emplace_back, try_push_back, try_emplace_back, and the unchecked versions are acceptable.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
9	7	0	0	0

- **POLL**: We approve the design of D0843R7 (inplace_vector) with the changes already polled.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
11	5	0	0	0

Standalone or a special case another type?

The EASTL (https://github.com/questor/eastl/blob/master/fixed_vector.h#L71) [2] and Folly (https://github.com/facebook/folly/blob/master/folly/docs/small_vector.md) [3] special case `small_vector`, e.g., using a fourth template parameter, to make it become an `inplace_vector`. P0639R0: *Changing attack vector of the* `constexpr_vector` [7] (http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0639r0.html) proposes improving the `Allocator` concepts to allow implementing `inplace_vector` as a special case of `vector` with a custom allocator. Both approaches produce specializations of `small_vector` or `vector` whose methods differ subtly in terms of effects, exception safety, iterator invalidation, and complexity guarantees.

This proposal closely follows `boost::container::static_vector<T,Capacity>` [1] (http://www.boost.org/doc/libs/1_59_0/doc/html/boost/container/static_vector.html) and proposes `inplace_vector` as a standalone type.

Where possible, this proposal defines the semantics of `inplace_vector` to match `vector`. Providing the same programming model makes this type easier to teach and use, and makes it easy to "just change" one type in a program to, e.g., perform a performance experiment without accidentally introducing undefined behavior.

Layout

`inplace_vector` models `ContiguousContainer`. Its elements are stored and properly aligned within the `inplace_vector` object itself. If the `Capacity` is zero the container has zero size:

```
static_assert(is_empty_v<inplace_vector<T, 0>>); // for all T
```

The offset of the first element within `inplace_vector` is unspecified, and `T` s are not allowed to overlap.

The layout differs from `vector` , since `inplace_vector` does not store the `capacity` field (it's known from the template parameter).

If `T` is trivially-copyable or `N == 0` , then `inplace_vector<T, N>` is also trivially copyable to support high-performance computing (HPC) use cases, such as the following.

- Copying between host and accelerator memory spaces. Examples of accelerators include Graphics Processing Units (GPUs).
- Serialization and deserialization for distributed-memory parallel communication, e.g., sending a vector via the `MPI_Send` function from the Message Passing Interface (MPI).

```
// for all C:
static_assert(!is_trivially_copyable_v<T> || is_trivially_copyable_v<inplace_vecto
```

Move semantics

A moved-from `inplace_vector` is left in a *valid but unspecified state* (option 3 below) unless `T` is trivially-copyable, in which case the size of the `inplace_vector` does not change (`array` semantics, option 2 below). That is:

> ```
> inplace_vector a(10);
> inplace_vector b(std::move(a));
> assert(a.size() == 10); // MAY FAIL
> ```

moves `a` 's elements element-wise into `b` , and afterwards the size of the moved-from `inplace_vector` may have changed.

This prevents code from relying on the size staying the same (and therefore being incompatible with changing an `inplace_vector` type back to `vector`) without incuring the cost of having to clear the `inplace_vector` .

When `T` is trivially-copyable, `array` semantics are used to provide trivial move operations.

This is different from LEWG Kona '22 Polls (https://github.com/cplusplus/papers/issues/114#issuecomment-1312015872) (22 in person + 8 remote) and we'd like to poll on these semantics again:

- **POLL**: Moving a static_vector should empty it (vector semantics).

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
9	10	4	2	2

- **POLL**: Moving a static_vector should leave it in a valid but unspecified state.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
6	9	1	5	6

Alternatives:

1. `vector` semantics: guarantees that `inplace_vector` is left empty (this happens with move assignment when using `std::allocator<T>` and always with move construction).
   - Pro: same programming model as `vector`.
   - Pro: increases safety by requiring users to re-initialize vector elements.
   - Con: clearing an `inplace_vector` is not free.
   - Con: `inplace_vector<T, N>` can no longer be made trivially copyable for a trivially copyable `T`, as the move operations can no longer be trivial.
2. `array` semantics: guarantees that `size()` of `inplace_vector` does not change, and that elements are left in their moved-from state.
   - Pro: no additional run-time cost incurred.
   - Con: different programming model than `vector`.
3. "valid but unspecified state"
   - Con: different programming model than `vector` and `array`, requires calling `size()`
   - Pro: code calling `size()` is correct for both `vector` and `inplace_vector`, enabling changing the type back and forth.

`constexpr` support

> Note: this revision brings back this section which was dropped in R8 because per LEWG discussion, LEWG thought that this whole type was implementable within constexpr for all types. The design intent of LEWG was to make it "as `constexpr` as possible."

The API of `inplace_vector<T, Capacity>` can be used in constexpr-contexts if `is_trivially_copyable_v<T>`, `is_default_constructible_v<T>`, and `is_trivially_destructible<T>` are true. This proposal only supports using the `constexpr` methods in constant expressions if `is_trivial_t<T>` is true.

The implementation cost of this is small. The prototye implementation specializes the storage to use a C array with value-initialized elements.

This negatively impacts the algorithmic complexity of inplace_vector constructors for these types from O(size) to O(capacity). When value-initialization takes place at run-time, this difference is significant.

Vectors with large capacity requirements are better served by vector instead.

Exception Safety

When using the `inplace_vector` APIs, the following types of failures are expected:

- May throw:
  1. The `value_type`'s constructors/assignment/destructors/swap (depends on `noexcept`),
  2. Mutating operations exceeding the capacity (`push_back`, `insert`, `inplace_vector(value_type, size)`, `inplace_vector(begin, end)` ...), and
  3. Out-of-bounds checked access: `at`.
- Pre-condition violation:
  1. Out-of-bounds unchecked access: `front` / `back` / `pop_back` when empty, `operator[]`.

Exception Safety guarantees of Mutating Operations

When an `inplace_vector` API throws an exception,

- *Basic Exception Guarantee* requires the API to leave the `inplace_vector` in a valid state.
- *Strong Exception Guarantee* requires the API to roll back the `inplace_vector` state to that of before the API was called, e.g., removing previously inserted elements, and loosing data when inserting from input iterators or ranges.

The following alternative were considered:

1. Same guarantees as their counter-part `vector` APIs.

2. Always provide the *Basic Guarantee* independent on the concepts implemented by the iterators/ranges: always insert up to the capacity, then throw.
3. Provide different exception safety guarantees depending on the concepts modeled by the iterators/ranges API arguments:
   - `sized_range`, `random_access_iterator`, or `LegacyRandomAccessIterator`: *Strong guarantee*, i.e., if the capacity would be exceeded, the API throws without attempting to insert any elements. This performs well and the caller looses no data.
   - Otherwise: *Basic guarantee*, i.e., elements are inserted up to the capacity, and are not removed before throwing. This performs well and the caller only looses data, e.g., stashed in discarded input iterators.

We propose to, unless stated otherwise, `inplace_vector` APIs should provide the same exception safety guarantees as their counter-part `vector` APIs.

Exception thrown by mutating operations exceeding capacity

We propose that mutating operations that exceed the capacity throw `bad_alloc`, to make it safer for applications handling out of memory errors to introduce `inplace_vector` as a performance optimization by replacing `vector`.

LEWG revisited the rationale below and decided to keep throwing `bad_alloc` in the 2024-01-30 telecon.

Alternatives:

1. Throw `bad_alloc`: `inplace_vector` requests storage from "allocator embedded within the `inplace_vector`", which fails to allocate, and therefore throws `bad_alloc` (e.g. like `vector` and `pmr` "stack allocator").
   - **Pros**: handling `bad_alloc` is more common than other exceptions when attempting to handle failure to insert due to "out-of-memory".
2. Throw `length_error`: insertion exceeds `max_size` and therefore throws `length_error`
   - **Pros**: container requirements already imply that this exception may be thrown.
   - **Cons**: handling `length_error` is rare since it is usually very high.
3. Throw "some other exception" when the `inplace_vector` is out-of-memory:
   - **Pros**: to be determined.
   - **Cons**: different programming model as `vector`.
4. Abort the process
   - **Pros**: portability to embedded platforms without exception support
   - **Cons**: different programming model than `vector`
5. Precondition violation

- **Cons**: different proramming model than `vector`, users responsible for checking before modifying vector size, etc.

Fallible APIs

We add the following new fallible APIs which, when the vector size equal its capacity, return `nullptr` (and do not throw `bad_alloc`) without moving from the inputs, enabling them to be re-used:

```
constexpr T* inplace_vector<T, C>::try_push_back(const T& value);
constexpr T* inplace_vector<T, C>::try_push_back(T&& value);

template<class... Args>
  constexpr T* try_emplace_back(Args&&... args);

template< container-compatible-range<T> R>
  constexpr ranges::iterator_t<R> try_append_range(R&& rg);
```

The `try_append_range` API always tries to insert all `rg` range elements up to either the vector capacity or the range `rg` is exhausted. It returns an iterator to the first non-inserted element of `rg` or the end iterator of `rg` if the range was exhausted. It intentionally provides the Basic Exception Safety guarantee, i.e., if inserting an element throws, previously succesfully inserted elements are preserved in the vector (i.e. not lost).

These APIs may be used as follows:

```
T value = T();
if (!v.try_push_back(value)) {
    std::cerr << "Failed to insert " << value << std::endl; // value not mov
    std::terminate();
}

auto il = {1, 2, 3};
if (v.try_append_range(il) != end(il)) {
    // The vector capacity was exhausted
    std::terminate();
}
```

Fallible Unchecked APIs

We add the following new fallible unchecked APIs for which exceeding the capacity is a precondition violation:

```
constexpr T& inplace_vector<T, C>::unchecked_push_back(const T& value);
constexpr T& inplace_vector<T, C>::unchecked_push_back(T&& value);

template<class... Args>
  constexpr T& unchecked_emplace_back(Args&&... args);
```

The `append_range` API was requested during LWG review in December 2023.

These APIs were requested in LEWG Kona '22 (https://github.com/cplusplus/papers/issues/114#issuecomment-1312015872) (22 in person + 8 remote):

- **POLL**: If static_vector has unchecked operations (e.g. `push_back_unchecked`), it is okay for checked operations (e.g. `push_back`) to throw when they run out of space.

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
14	4	2	4	1

This was confirmed at LEWG Varna '23 () after a discussion on safety:

- **POLL**: D0843R7 should remove the unchecked versions of push_back and emplace_back

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
1	5	3	3	7

The name `unchecked_push_back` was polled in LEWG Varna '23 ():

- **POLL**: (vote for all the options you find acceptable, vote as many times as you like) Feature naming

Feature name	Votes
push_back_unchecked	11
unchecked_push_back	16
unsafe_push_back	9
push_back_unsafe	6

The potential impact of the three APIs on code size and performance is shown here (https://clang.godbolt.org/z/MbG17q8x1), where the main difference between `try_push_back` and `unchecked_push_back` is the presence of an extra branch in `try_push_back`.

Allocator awareness

We believe that right now, making `inplace_vector` allocator-aware does not outweigh its complexity and design cost. We can always provide a way to support that in the future.

Options:

- `inplace_vector` is allocator-aware if its `value_type` is allocator-aware.
- factoring an allocator-aware `inplace_vector` into a separate `basic_allocator` class.
- no support for now (not worth delaying further)

Iterator invalidation

`inplace_vector` iterator invalidation guarantees differ from `std::vector`:

- moving a `inplace_vector` invalidates all iterators, and
- swapping two `inplace_vector`s invalidates all iterators.

`inplace_vector` APIs that potentially invalidate iterators are: `resize(n)`, `resize(n, v)`, `pop_back`, `erase`, and `swap`.

Freestanding

Many `inplace_vector` APIs are not available in freestanding because fallible insertion APIs (constructors, push back, insert, ...) may throw.

The infallible `try_` APIs do not throw and are available in freestanding. They only cover a subset of the functionality available through fallible APIs. This is intentional. Adding more infallible APIs to `inplace_vector` and potentially other containers is left as future work.

We'd need to add it to: [library.requirements.organization.compliance] (http://eel.is/c++draft/compliance)

When we fix this we'd need to add `<inplace_vector>` to [tab:headers.cpp.fs] (http://eel.is/c++draft/tab:headers.cpp.fs):

	Subclause	Headers
[containers] (http://eel.is/c++draft/containers)	containers	`<inplace_vector>`

Same or Separate header

We propose that this container goes into its own header `<inplace_vector>` rather than in header `<vector>`, because it is a sufficiently different container.

LWG asked for `inplace_vector` to be part of the `<vector>` header. LEWG Varna '23 () took the following poll:

- **POLL:** D0843R7 should provide `inplace_vector` in `<vector>` rather than the proposal's decision on `<inplace_vector>`

Strongly Favor	Weakly Favor	Neutral	Weakly Against	Strongly Against
0	0	1	12	5

That is, consensus against change.

Return type of push_back

In C++20, both `push_back` and `emplace_back` were slated to return a `reference` (they used to both return `void`). Even with plenary approval, changing `push_back` turned out to be an ABI break that was backed out, leaving the situation where `emplace_back` returns a `reference` but `push_back` is still `void`. This ABI issue doesn't apply to new types. Should `push_back` return a `reference` to be consistent with `emplace_back`, or should it be consistent with older containers?

Request LEWG to poll on that.

`reserve` and `shrink_to_fit` APIs

`shrink_to_fit` requests `vector` to decrease its `capacity`, but this request may be ignored. `inplace_vector` may implement it as a nop (and it may be `noexcept`).

`reserve(n)` requests the `vector` to potentially increase its `capacity`, failing if the request can't be satisfied. `inplace_vector` may implement it as a nop if `n <= capacity()`, throwing `bad_alloc` otherwise.

These APIs make it easier and safe for programs to be "more" parametric over "vector-like" containers (`vector`, `small_vector`, `inplace_vector`), but since they do not do anything useful for `inplace_vector`, we may want to fail to compile instead.

Deduction guides

Unlike the other containers, `inplace_vector` does not have any deduction guides because there is no case in which it would be possible to deduce the second template argument, the capacity, from the initializer.

Summary of semantic differences with `vector`

Aspect	**`vector`**	**`inplace_vector`**
Capacity	Indefinite	`N`
Move and swap	O(1), no iterators invalidated	`array` semantics: O(size), invalidates all iterators
Moved from	left empty (this happens with move assignment when using std::allocator<T> and always with move construction)	valid but unspecified state except if `T` is trivially-copyable, in which case `array` semantics
Default construction and destruction of trivial types	O(1)	O(capacity)
Is empty when zero capacity?	No	Yes
Trivially-copyable if `is_trivially_copyable_v<T>` ?	No	Yes

Name

The class template name was confirmed at LEWG Varna '23 ():

- **POLL**: Feature naming

Options	**Votes**
static_vector	4
inplace_vector	14
fixed_capacity_vector	5

Technical specification

EDITORIAL: This enhancement is a pure header-only addition to the C++ standard library as the `<inplace_vector>` header. It belongs in the "Sequence containers" ([sequences] (http://eel.is/c++draft/sequences)) part of the "Containers library" ([containers] (http://eel.is/c++draft/containers)) as "Class template `inplace_vector`".

[library.requirements.organization.headers] (http://eel.is/c++draft/headers)

Add `<inplace_vector>` to [tab:headers.cpp] (http://eel.is/c++draft/tab:headers.cpp).

Add `<inplace_vector>` to [tab:headers.cpp.fs] (http://eel.is/c++draft/tab:headers.cpp.fs):

	Subclause	Headers
[containers] (http://eel.is/c++draft/containers)	containers	`<inplace_vector>`

[iterator.range] (https://eel.is/c++draft/iterator.range) Range access

Modify:

1 (https://eel.is/c++draft/iterator.range#1) In addition to being available via inclusion of the `<iterator>` header, the function templates in [iterator.range] (https://eel.is/c++draft/iterator.range) are available when any of the following headers are included: `<array>`, `<deque>`, `<forward_list>`, `<inplace_vector>`, `<list>`, `<map>`, `<regex>`, `<set>`, `<span>`, `<string>`, `<string_view>`, `<unordered_map>`, `<unordered_set>`, and `<vector>`.

[container.alloc.reqmts] (http://eel.is/c++draft/container.alloc.reqmts)

Modify:

1 (http://eel.is/c++draft/container.alloc.reqmts#1) All of the containers defined in [containers] (http://eel.is/c++draft/containers) and in [basic.string] (http://eel.is/c++draft/basic.string) except `array` and `inplace_vector` meet the additional requirements of an allocator-aware container, as described below.

[allocator.requirements.general] (https://eel.is/c++draft/allocator.requirements.general)

1 (https://eel.is/c++draft/allocator.requirements.general#1) The library describes a standard set of requirements for *allocators*, which are class-type objects that encapsulate the information about an allocation model. This information includes the knowledge of pointer types, the type of their difference, the type of the size of objects in this allocation model, as well as the

memory allocation and deallocation primitives for it. All of the string types, containers (except `array` and `inplace_vector`), string buffers and string streams ([input.output] (https://eel.is/c++draft/input.output)), and match_results are parameterized in terms of allocators.

[containers.general] (http://eel.is/c++draft/containers.general)

Modify [tab:containers.summary] (http://eel.is/c++draft/tab:containers.summary):

	Subclause	Headers
[sequences] (http://eel.is/c++draft/sequences)	Sequence containers	<array>, <deque>, <forward_list>, <inplace_vector>, <list>, <vector>

[container.reqmts] (http://eel.is/c++draft/container.reqmts) General container requirements

1. A type `X` meets the container requirements if the following types, statements, and expressions are well-formed and have the specified semantics.

`typename X::value_type`

- Result: `T`
- Preconditions: `T` is `Cpp17Erasable` from `X` (see [container.alloc.reqmts], below).

`typename X::reference`

- Result: `T&`

`typename X::const_reference`

- Result: `const T&`

`typename X::iterator`

- Result: A type that meets the forward iterator requirements ([forward.iterators]) with value type `T`. The type `X::iterator` is convertible to `X::const_iterator`.

`typename X::const_iterator`

- Result: A type that meets the requirements of a constant iterator and those of a forward iterator with value type `T`.

`typename X::difference_type`

- Result: A signed integer type, identical to the difference type of `X::iterator` and `X::const_iterator`.

`typename X::size_type`

- Result: An unsigned integer type that can represent any non-negative value of `X::difference_type`.

```
X u;
X u = X();
```

- Postconditions: `u.empty()`
- Complexity: Constant.

```
X u(a);
X u = a;
```

- Preconditions: `T` is `Cpp17CopyInsertable` into `X` (see below).
- Postconditions: `u == a`
- Complexity: Linear.

```
X u(rv);
X u = rv;
```

- Postconditions: `u` is equal to the value that `rv` had before this construction.
- Complexity: Linear for array and `inplace_vector` and constant for all other standard containers.

`a = rv`

- Result: `X&`.
- Effects: All existing elements of `a` are either move assigned to or destroyed.
- Postconditions: If `a` and `rv` do not refer to the same object, `a` is equal to the value that `rv` had before this assignment.
- Complexity: Linear.

`a.~X()`

- Result: `void`
- Effects: Destroys every element of `a`; any memory obtained is deallocated.
- Complexity: Linear.

`a.begin()`

- Result: `iterator`; `const_iterator` for constant `a`.
- Returns: An iterator referring to the first element in the container.
- Complexity: Constant.

`a.end()`

- Result: `iterator`; `const_iterator` for constant `a`.
- Returns: An iterator which is the past-the-end value for the container.
- Complexity: Constant.

`a.cbegin()`

- Result: `const_iterator`.
- Returns: `const_cast<X const&>(a).begin()`
- Complexity: Constant.

`a.cend()`

- Result: `const_iterator`.
- Returns: `const_cast<X const&>(a).end()`
- Complexity: Constant.

`i <=> j`

- Result: `strong_ordering` .
- Constraints: `X::iterator` meets the random access iterator requirements.
- Complexity: Constant.

`a == b`

- Preconditions: `T` meets the `Cpp17EqualityComparable` requirements.
- Result: Convertible to `bool` .
- Returns: `equal(a.begin(), a.end(), b.begin(), b.end())` [Note 1: The algorithm `equal` is defined in [alg.equal]. — end note]
- Complexity: Constant if `a.size() != b.size()` , linear otherwise.
- Remarks: `==` is an equivalence relation.

`a != b`

- Effects: Equivalent to `!(a == b)` .

`a.swap(b)`

- Result: `void`
- Effects: Exchanges the contents of `a` and `b` .
- Complexity: Linear for array and `inplace_vector` , and constant for all other standard containers.

`swap(a, b)`

- Effects: Equivalent to `a.swap(b)` .

`r = a`

- Result: `X&` .
- Postconditions: `r == a` .
- Complexity: Linear.

`a.size()`

- Result: `size_type` .
- Returns: `distance(a.begin(), a.end())` , i.e. the number of elements in the container.
- Complexity: Constant.
- Remarks: The number of elements is defined by the rules of constructors, inserts, and erases.

`a.max_size()`

- Result: `size_type` .
- Returns: `distance(begin(), end())` for the largest possible container.
- Complexity: Constant.

`a.empty()`

- Result: Convertible to `bool` .
- Returns: `a.begin() == a.end()`
- Complexity: Constant.
- Remarks: If the container is empty, then `a.empty()` is `true` .

1. In the expressions

```
i == j
i != j
i < j
i <= j
i >= j
i > j
i <=> j
i - j
```

where `i` and `j` denote objects of a container's iterator type, either or both may be replaced by an object of the container's `const_iterator` type referring to the same element with no change in semantics.

Unless otherwise specified, all containers defined in this Clause obtain memory using an allocator (see [allocator.requirements] (https://eel.is/c++draft/allocator.requirements)).

[Note 2: In particular, containers and iterators do not store references to allocated elements other than through the allocator's pointer type, i.e., as objects of type `P` or `pointer_traits<P>::template rebind<unspecified>`, where `P` is `allocator_traits<allocator_type>::pointer`. — end note]

Copy constructors for these container types obtain an allocator by calling `allocator_traits<allocator_type>::select_on_container_copy_construction` on the allocator belonging to the container being copied. Move constructors obtain an allocator by move construction from the allocator belonging to the container being moved. Such move construction of the allocator shall not exit via an exception. All other constructors for these container types take a `const allocator_type&` argument.

[Note 3: If an invocation of a constructor uses the default value of an optional allocator argument, then the allocator type must support value-initialization. — end note]

A copy of this allocator is used for any memory allocation and element construction performed, by these constructors and by all member functions, during the lifetime of each container object or until the allocator is replaced. The allocator may be replaced only via assignment or `swap()`. Allocator replacement is performed by copy assignment, move assignment, or swapping of the allocator only if

1. `allocator_traits<allocator_type>::propagate_on_container_copy_assignment::value`,
2. `allocator_traits<allocator_type>::propagate_on_container_move_assignment::value`, or
3. `allocator_traits<allocator_type>::propagate_on_container_swap::value`

   is `true` within the implementation of the corresponding container operation. In all container types defined in this Clause, the member `get_allocator()` returns a copy of the allocator used to construct the container or, if that allocator has been replaced, a copy of the most recent replacement.

The expression `a.swap(b)`, for containers `a` and `b` of a standard container type other than `array` and `inplace_vector`, shall exchange the values of `a` and `b` without invoking any move, copy, or swap operations on the individual container elements. Lvalues of any Compare, Pred, or Hash types belonging to `a` and `b` shall be swappable and shall be exchanged by calling swap as described in [swappable.requirements] (http://eel.is/c++draft/swappable.requirements). If `allocator_traits<allocator_type>::propagate_on_container_swap::value` is `true`, then lvalues of type `allocator_type` shall be swappable and the allocators of `a` and `b` shall also be exchanged by calling `swap` as described in [swappable.requirements] (http://eel.is/c++draft/swappable.requirements). Otherwise, the allocators shall not be swapped, and the behavior is undefined unless `a.get_allocator() == b.get_allocator()`. Every iterator

referring to an element in one container before the swap shall refer to the same element in the other container after the swap. It is unspecified whether an iterator with value `a.end()` before the swap will have value `b.end()` after the swap.

Unless otherwise specified (see [associative.reqmts.except], [unord.req.except], [deque.modifiers], [inplace.vector.modifiers] and [vector.modifiers]) all container types defined in this Clause meet the following additional requirements:

1. If an exception is thrown by an insert() or emplace() function while inserting a single element, that function has no effects.
2. If an exception is thrown by a push_back(), push_front(), emplace_back(), or emplace_front() function, that function has no effects.
3. No erase(), clear(), pop_back() or pop_front() function throws an exception.
4. No copy constructor or assignment operator of a returned iterator throws an exception.
5. No swap() function throws an exception.
6. No swap() function invalidates any references, pointers, or iterators referring to the elements of the containers being swapped.
   [Note 4: The end() iterator does not refer to any element, so it can be invalidated. — end note]

[containers.sequences.general] (http://eel.is/c++draft/sequences.general)

Modify:

1 (http://eel.is/c++draft/sequences.general#1) The headers `<array>`, `<deque>`, `<forward_list>`, `<inplace_vector>`, `<list>`, and <vector> define class templates that meet the requirements for sequence containers.

[container.requirements.sequence.reqmts]

(http://eel.is/c++draft/sequence.reqmts)

Modify:

sequence.reqmts.1 (http://eel.is/c++draft/sequence.reqmts#1) A sequence container organizes a finite set of objects, all of the same type, into a strictly linear arrangement. The library provides ~~four~~the following basic kinds of sequence containers: `vector`, `inplace_vector`, `forward_list`, `list`, and `deque`. In addition, `array` is provided as a sequence container which provides limited sequence operations because it has a fixed number of elements. The library also provides container adaptors that make it easy to construct abstract data types,

such as `stacks`, `queues`, `flat_maps`, `flat_multimaps`, `flat_sets`, or `flat_multisets`, out of the basic sequence container kinds (or out of other program-defined sequence containers).

sequence.reqmts.2 (http://eel.is/c++draft/sequence.reqmts#2) [Note 1 (http://eel.is/c++draft/sequence.reqmts#note-1): The sequence containers offer the programmer different complexity trade-offs. `vector` is appropriate in most circumstances. `array` has a fixed size known during translation. `inplace_vector` has a fixed capacity known during translation. `list` or `forward_list` support frequent insertions and deletions from the middle of the sequence. `deque` supports efficient insertions and deletions taking place at the beginning or at the end of the sequence. When choosing a container, remember `vector` is best; leave a comment to explain if you choose from the rest! — end note]

seqeuence.reqmts.5 (https://eel.is/c++draft/container.requirements#sequence.reqmts-5)

```
X u(n, t);
```

- Preconditions: T is Cpp17CopyInsertable into X.
- Effects: Constructs a sequence container with n copies of t.
- Postconditions: distance(u.begin(), u.end()) == n is `true`.

```
X u(i, j);
```

- Preconditions: T is Cpp17EmplaceConstructible into X from *i. For vector, if the iterator does not meet the Cpp17ForwardIterator requirements ([forward.iterators]), T is also Cpp17MoveInsertable into X.
- Effects: Constructs a sequence container equal to the range [i, j). Each iterator in the range [i, j) is dereferenced exactly once.
- Postconditions: distance(u.begin(), u.end()) == distance(i, j) is `true`.

```
X(from_range, rg)
```

- Preconditions: T is Cpp17EmplaceConstructible into X from `*ranges::begin(rg)`. For vector, if R models neither `ranges::sized_range` nor `ranges::forward_range`, T is also Cpp17MoveInsertable into X.
- Effects: Constructs a sequence container equal to the range rg. Each iterator in the range rg is dereferenced exactly once.
- Postconditions: `distance(begin(), end()) == ranges::distance(rg)` is true.

`X(il)`

- Effects: Equivalent to X(il.begin(), il.end()).

`a = il`

- Result: `X&` .
- Preconditions: T is Cpp17CopyInsertable into X and Cpp17CopyAssignable.
- Effects: Assigns the range [il.begin(), il.end()) into a. All existing elements of a are either assigned to or destroyed.
- Returns: *this.

`a.emplace(p, args)`

- Result: iterator.
- Preconditions: T is Cpp17EmplaceConstructible into X from args. For vector and deque, T is also Cpp17MoveInsertable into X and Cpp17MoveAssignable.
- Effects: Inserts an object of type T constructed with `std::forward<Args>(args)...` before p.
  [Note 1: args can directly or indirectly refer to a value in a. — end note]
- Returns: An iterator that points to the new element constructed from args into a.

`a.insert(p, t)`

- Result: iterator.
- Preconditions: T is Cpp17CopyInsertable into X. For vector, inplace_vector and deque, T is also Cpp17CopyAssignable.
- Effects: Inserts a copy of t before p.
- Returns: An iterator that points to the copy of t inserted into a.

`a.insert(p, rv)`

- Result: iterator.
- Preconditions: T is Cpp17MoveInsertable into X. For vector, inplace_vector and deque, T is also Cpp17MoveAssignable.
- Effects: Inserts a copy of rv before p.

- Returns: An iterator that points to the copy of rv inserted into a.

`a.insert(p, n, t)`

- Result: iterator.
- Preconditions: T is Cpp17CopyInsertable into X and Cpp17CopyAssignable.
- Effects: Inserts n copies of t before p.
- Returns: An iterator that points to the copy of the first element inserted into a, or p if n == 0.

`a.insert(p, i, j)`

- Result: iterator.
- Preconditions: T is Cpp17EmplaceConstructible into X from *i. For vector, inplace_vector and deque, T is also Cpp17MoveInsertable into X, and T meets the Cpp17MoveConstructible, Cpp17MoveAssignable, and Cpp17Swappable ([swappable.requirements]) requirements. Neither i nor j are iterators into a.
- Effects: Inserts copies of elements in [i, j) before p. Each iterator in the range [i, j) shall be dereferenced exactly once.
- Returns: An iterator that points to the copy of the first element inserted into a, or p if i == j.

`a.insert_range(p, rg)`

- Result: iterator.
- Preconditions: T is Cpp17EmplaceConstructible into X from `*ranges::begin(rg)`. For vector, inplace_vector and deque, T is also Cpp17MoveInsertable into X, and T meets the Cpp17MoveConstructible, Cpp17MoveAssignable, and Cpp17Swappable ([swappable.requirements]) requirements. rg and a do not overlap.
- Effects: Inserts copies of elements in rg before p. Each iterator in the range rg is dereferenced exactly once.
- Returns: An iterator that points to the copy of the first element inserted into a, or p if rg is empty.

`a.insert(p, il)`

- Effects: Equivalent to a.insert(p, il.begin(), il.end()).

`a.erase(q)`

- Result: iterator.
- Preconditions: For vector, inplace_vector and deque, T is Cpp17MoveAssignable.
- Effects: Erases the element pointed to by q.
- Returns: An iterator that points to the element immediately following q prior to the element being erased. If no such element exists, a.end() is returned.

`a.erase(q1, q2)`

- Result: iterator.
- Preconditions: For vector, inplace_vector and deque, T is Cpp17MoveAssignable.
- Effects: Erases the elements in the range [q1, q2).
- Returns: An iterator that points to the element pointed to by q2 prior to any elements being erased. If no such element exists, a.end() is returned.

`a.clear()`

- Result: void
- Effects: Destroys all elements in a. Invalidates all references, pointers, and iterators referring to the elements of a and may invalidate the past-the-end iterator.
- Postconditions: a.empty() is true.
- Complexity: Linear.

`a.assign(i, j)`

- Result: void
- Preconditions: T is Cpp17EmplaceConstructible into X from *i and assignable from *i. For vector, if the iterator does not meet the forward iterator requirements ([forward.iterators]), T is also Cpp17MoveInsertable into X. Neither i nor j are iterators into a.
- Effects: Replaces elements in a with a copy of [i, j). Invalidates all references, pointers and iterators referring to the elements of a. For vector and deque, also invalidates the past-the-end iterator. Each iterator in the range [i, j) is dereferenced exactly once.

`a.assign_range(rg)`

- Result: void
- Mandates: assignable_from<T&, ranges::range_reference_t<R>> is modeled.
- Preconditions: T is Cpp17EmplaceConstructible into X from `*ranges::begin(rg)`. For vector, if R models neither `ranges::sized_range` nor `ranges::forward_range`, T is also Cpp17MoveInsertable into X. rg and a do not overlap.
- Effects: Replaces elements in a with a copy of each element in rg. Invalidates all references, pointers, and iterators referring to the elements of a. For vector and deque, also invalidates the past-the-end iterator. Each iterator in the range rg is dereferenced exactly once.

`a.assign(il)`

- Effects: Equivalent to a.assign(il.begin(), il.end()).

`a.assign(n, t)`

- Result: void
- Preconditions: T is Cpp17CopyInsertable into X and Cpp17CopyAssignable. t is not a reference into a.
- Effects: Replaces elements in a with n copies of t. Invalidates all references, pointers and iterators referring to the elements of a. For vector and deque, also invalidates the past-the-end iterator.

sequence.reqmts.69 (http://eel.is/c++draft/sequence.reqmts#69) The following operations are provided for some types of sequence containers but not others. An implementation shall implement them so as to take amortized constant time.

`a.front()`

- Result: `reference`; `const_reference` for constant `a`.
- Returns: `*a.begin()`
- Remarks: Required for `basic_string`, `array`, `deque`, `forward_list`, `inplace_vector`, `list`, and vector.

`a.back()`

- Result: `reference` ; `const_reference` for constant `a` .
- Effects: Equivalent to:

```
auto tmp = a.end();
--tmp;
return *tmp;
```

- Remarks: Required for `basic_string` , `array` , `deque` , `inplace_vector` , `list` , and vector.

`a.emplace_front(args)`

- Result: `reference`
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `args` .
- Effects: Prepends an object of type `T` constructed with `std::forward<Args>(args)...``.
- Returns: `a.front()``.
- Remarks: Required for `deque` , `forward_list` , and `list` .

`a.emplace_back(args)`

Drafting note: `inplace_vector` is never reallocated, so there is no need to extend the "For vector, T is also Cpp17MoveInsertable into X" to inplace_vector.

Drafting note: It's okay to use `Cpp17MoveInsertable` here, even though `inplace_vector` isn't allocator-aware. [container.alloc.reqmts.2] states: "If `X` is not allocator-aware or is a specialization of `basic_string` , the terms below [including `Cpp17MoveInsertable`] are defined as if `A` were allocator<T>".

- Result: reference
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `args` . For `vector` , `T` is also `Cpp17MoveInsertable` into `X` .
- Effects: Appends an object of type `T` constructed with `std::forward<Args>(args)...` .
- Returns: `a.back()``.
- Remarks: Required for `deque` , `inplace_vector` , `list` , and vector.

`a.push_front(t)`

- Result: `void`
- Preconditions: `T` is `Cpp17CopyInsertable` into `X`.
- Effects: Prepends a copy of `t`.
- Remarks: Required for `deque`, `forward_list`, and `list`.

`a.push_front(rv)`

- Result: `void`
- Preconditions: `T` is `Cpp17MoveInsertable` into `X`.
- Effects: Prepends a copy of `rv`.
- Remarks: Required for `deque`, `forward_list`, and `list`.

`a.prepend_range(rg)`

- Result: `void`
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `*ranges::begin(rg)`.
- Effects: Inserts copies of elements in `rg` before `begin()`. Each iterator in the range `rg` is dereferenced exactly once. [Note 3: The order of elements in `rg` is not reversed. — end note]
- Remarks: Required for `deque`, `forward_list`, and `list`.

`a.push_back(t)`

- Result: Reference
- Returns: `a.back()`.
- Preconditions: `T` is `Cpp17CopyInsertable` into `X`.
- Effects: Appends a copy of `t`.
- Remarks: Required for `basic_string`, `deque`, `inplace_vector`, `list`, and vector.

`a.push_back(rv)`

- Result: Reference
- Returns: `a.back()`

- Preconditions: `T` is `Cpp17MoveInsertable` into `X`.
- Effects: Appends a copy of `rv`.
- Remarks: Required for `basic_string`, `deque`, `inplace_vector`, `list`, and vector.

`a.append_range(rg)`

- Result: `void`
- Preconditions: `T` is `Cpp17EmplaceConstructible` into `X` from `*ranges::begin(rg)`. For `vector`, `T` is also `Cpp17MoveInsertable` into `X`.
- Effects: Inserts copies of elements in `rg` before `end()`. Each iterator in the range `rg` is dereferenced exactly once.
- Remarks: Required for `deque`, `inplace_vector`, `list`, and vector.

`a.pop_front()`

- Result: `void`
- Preconditions: `a.empty()` is `false`.
- Effects: Destroys the first element.
- Remarks: Required for `deque`, `forward_list`, and `list`.

`a.pop_back()`

- Result: `void`
- Preconditions: `a.empty()` is `false`.
- Effects: Destroys the last element.
- Remarks: Required for `basic_string`, `deque`, `inplace_vector`, `list`, and vector.

`a[n]`

- Result: `reference`; `const_reference` for constant `a`
- Returns: `*(a.begin() + n)`
- Remarks: Required for `basic_string`, `array`, `deque`, `inplace_vector`, and vector.

`a.at(n)`

- Result: `reference`; `const_reference` for constant `a`

- Returns: `*(a.begin() + n)`
- Throws: `out_of_range` if `n >= a.size()`.
- Remarks: Required for `basic_string`, `array`, `deque`, `inplace_vector`, and `vector`.

[containers.sequences.inplace.vector.syn] Header `<inplace_vector>` synopsis

EDITORIAL: this synopsis goes after the corresponding one from `<vector>` : [vector.syn] (https://eel.is/c++draft/vector.syn).

```
// mostly freestanding
#include <compare>              // see [compare.syn]
#include <initializer_list>     // see [initializer.list.syn]

namespace std {

// [inplace.vector], class template inplace_vector
template <class T, size_t N>
class inplace_vector; // partially freestanding

// [inplace.vector.erasure], erasure
template<class T, size_t N, class U>
constexpr typename inplace_vector<T, N>::size_type
  erase(inplace_vector<T, N>& c, const U& value);
template<class T, size_t N, class Predicate>
constexpr typename inplace_vector<T, N>::size_type
  erase_if(inplace_vector<T, N>& c, Predicate pred);

}  // namespace std
```

Drafting note: `erase` and `erase_if` are specified in terms of `remove` and `remove_if` which are not freestanding, and are therefore not freestanding here.

[containers.sequences.inplace.vector] Class template `inplace_vector`

EDITORIAL: this section goes after the corresponding one from `<vector>` : [vector] (https://eel.is/c++draft/vector). The sub-sections of this section are nested within it.

[containers.sequences.inplace.vector.overview] Overview

1. An `inplace_vector` is a contiguous container. Its capacity is fixed and its elements are stored within the `inplace_vector` object itself.
2. An `inplace_vector` meets all of the requirements of a container ([container.reqmts](http://eel.is/c++draft/container.reqmts)), of a reversible container ([container.rev.reqmts](http://eel.is/c++draft/container.rev.reqmts)), of a contiguous container, and of a sequence container, including most of the optional sequence container requirements ([sequence.reqmts](http://eel.is/c++draft/sequence.reqmts)). The exceptions are the `push_front`, `prepend_range`, `pop_front`, and `emplace_front` member functions, which are not provided. Descriptions are provided here only for operations on `inplace_vector` that are not described in one of these tables or for operations where there is additional semantic information.
3. For any `N`, `inplace_vector<T, N>::iterator` and `inplace_vector<T, N>::const_iterator` meet the constexpr iterator requirements.
4. For any `N > 0`, if `is_trivial_v<T>` is false, then no `inplace_vector<T, N>` member functions are usable in constant expressions.

LWG Review TODO: We need to review this last clause about constexpr. LEWG design direction is that `inplace_vector` should be "as `constexpr` as possible", i.e., it should be `constexpr` for the types for which C++26 can support `constexpr` here. That is, this clause needs to state that `constexpr` only applies to the types for which it's implementable in C++26. LWG should tell us whether this is implicit-lifetime types, trivial types, or something else, and how to word this requirement. The current proposal is to make the API conditionally constexpr.

5. Any member function of `inplace_vector<T, N>` that would cause the size to exceed `N` throws an exception of type `bad_alloc`.

Drafting note: this is required because `assign` and others come from the container requirements, and this extends that.

6. Let `IV` denote a specialization of `inplace_vector<T, N>`. If `N` is zero, then `IV` is both trivial and empty. Otherwise:
   - If `is_trivially_copy_constructible_v<T>` is `true`, then `IV` has a trivial copy constructor.
   - If `is_trivially_move_constructible_v<T>` is `true`, then `IV` has a trivial move constructor.
   - If `is_trivially_destructible_v<T>` is `true`, then:
     - `IV` has a trivial destructor,

- if `is_trivially_copy_constructible_v<T> && is_trivially_copy_assignable_v<T>` is `true`, then `IV` has a trivial copy assignment operator,
- if `is_trivially_move_constructible_v<T> && is_trivially_move_assignable_v<T>` is `true`, then `IV` has a trivial move assignment operator.

```
template <class T, size_t N>
class inplace_vector {
public:
// types:
using value_type             = T;
using pointer                = T*;
using const_pointer          = const T*;
using reference              = value_type&;
using const_reference        = const value_type&;
using size_type              = size_t;
using difference_type        = ptrdiff_t;
using iterator               = implementation-defined; // see [container.req
using const_iterator         = implementation-defined; // see [container.req
using reverse_iterator       = std::reverse_iterator<iterator>;
using const_reverse_iterator = std::reverse_iterator<const_iterator>;

// [containers.sequences.inplace.vector.cons], construct/copy/destroy
constexpr inplace_vector() noexcept;
constexpr explicit inplace_vector(size_type n); // freestanding-deleted
constexpr inplace_vector(size_type n, const T& value); // freestanding-delet
template <class InputIterator>
  constexpr inplace_vector(InputIterator first, InputIterator last); // free
template <container-compatible-range<T> R>
  constexpr inplace_vector(from_range_t, R&& rg); // freestanding-deleted
constexpr inplace_vector(const inplace_vector&);
constexpr inplace_vector(inplace_vector&&)
  noexcept(N == 0 || is_nothrow_move_constructible_v<T>);
constexpr inplace_vector(initializer_list<T> il); // freestanding-deleted
constexpr ~inplace_vector();
constexpr inplace_vector& operator=(const inplace_vector& other);
constexpr inplace_vector& operator=(inplace_vector&& other)
  noexcept(N == 0 || (is_nothrow_move_assignable_v<T> && is_nothrow_move_con
constexpr inplace_vector& operator=(initializer_list<T>); // freestanding-de
template <class InputIterator>
  constexpr void assign(InputIterator first, InputIterator last); // freesta
template<container-compatible-range<T> R>
  constexpr void assign_range(R&& rg); // freestanding-deleted
constexpr void assign(size_type n, const T& u); // freestanding-deleted
constexpr void assign(initializer_list<T> il); // freestanding-deleted

// iterators
constexpr iterator               begin()         noexcept;
constexpr const_iterator         begin()   const noexcept;
constexpr iterator               end()           noexcept;
constexpr const_iterator         end()     const noexcept;
constexpr reverse_iterator       rbegin()        noexcept;
constexpr const_reverse_iterator rbegin()  const noexcept;
constexpr reverse_iterator       rend()          noexcept;
constexpr const_reverse_iterator rend()    const noexcept;
```

```
constexpr const_iterator         cbegin()  const noexcept;
constexpr const_iterator         cend()    const noexcept;
constexpr const_reverse_iterator crbegin() const noexcept;
constexpr const_reverse_iterator crend()   const noexcept;

// [containers.sequences.inplace.vector.members] size/capacity
constexpr bool empty() const noexcept;
constexpr size_type size() const noexcept;
static constexpr size_type max_size() noexcept;
static constexpr size_type capacity() noexcept;
constexpr void resize(size_type sz); // freestanding-deleted
constexpr void resize(size_type sz, const T& c); // freestanding-deleted
static constexpr void reserve(size_type n); // freestanding-deleted
static constexpr void shrink_to_fit() noexcept;

// element access
constexpr reference       operator[](size_type n);
constexpr const_reference operator[](size_type n) const;
constexpr const_reference at(size_type n) const; // freestanding-deleted
constexpr reference       at(size_type n); // freestanding-deleted
constexpr reference       front();
constexpr const_reference front() const;
constexpr reference       back();
constexpr const_reference back() const;

// [containers.sequences.inplace.vector.data], data access
constexpr       T* data()       noexcept;
constexpr const T* data() const noexcept;

// [containers.sequences.inplace.vector.modifiers], modifiers
template <class... Args> constexpr reference emplace_back(Args&&... args); /
constexpr reference push_back(const T& x); // freestanding-deleted
constexpr reference push_back(T&& x); // freestanding-deleted
template<container-compatible-range<T> R>
  constexpr void append_range(R&& rg); // freestanding-deleted
constexpr void pop_back();

template<class... Args>
  constexpr pointer try_emplace_back(Args&&... args);
constexpr pointer try_push_back(const T& x);
constexpr pointer try_push_back(T&& x);
template<container-compatible-range<T> R>
  constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);

template<class... Args>
  constexpr reference unchecked_emplace_back(Args&&... args);
constexpr reference unchecked_push_back(const T& x);
constexpr reference unchecked_push_back(T&& x);

template <class... Args>
```

```
template <class... Args>
  constexpr iterator emplace(const_iterator position, Args&&... args); // fr
constexpr iterator insert(const_iterator position, const T& x); // freestand
constexpr iterator insert(const_iterator position, T&& x); // freestanding-d
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template <class InputIterator>
  constexpr iterator insert(const_iterator position, InputIterator first, In
template<container-compatible-range<T> R>
  constexpr iterator insert_range(const_iterator position, R&& rg); // frees
constexpr iterator insert(const_iterator position, initializer_list<T> il);
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void swap(inplace_vector& x)
  noexcept(N == 0 || (is_nothrow_swappable_v<T> && is_nothrow_move_construct
constexpr void clear() noexcept;

constexpr friend bool operator==(const inplace_vector& x, const inplace_vect
constexpr friend synth-three-way-result<T>
  operator<=>(const inplace_vector& x, const inplace_vector& y);
constexpr friend void swap(inplace_vector& x, inplace_vector& y)
  noexcept(N == 0 || (is_nothrow_swappable_v<T> && is_nothrow_move_construct
  { x.swap(y); }
};
```

[containers.sequences.inplace.vector.cons] Constructors

```
constexpr explicit inplace_vector(size_type n);
```

- *Preconditions*: `T` is `Cpp17DefaultInsertable` into `*this`.
- *Effects*: Constructs an `inplace_vector` with `n` default-inserted elements.
- *Complexity*: Linear in `n`.

```
constexpr inplace_vector(size_type n, const T& value);
```

- *Preconditions*: `T` is `Cpp17CopyInsertable` into `*this`.
- *Effects*: Constructs an `inplace_vector` with `n` copies of `value`.
- *Complexity*: Linear in `n`.

```
template <class InputIterator>
constexpr inplace_vector(InputIterator first, InputIterator last);
```

- *Effects*: Constructs an `inplace_vector` equal to the range [`first`, `last`).
- *Complexity*: Linear in `distance(first, last)`.

```
template <container-compatible-range<T> R>
constexpr inplace_vector(from_range_t, R&& rg);
```

- *Effects*: Constructs an `inplace_vector` object with the elements of the range `rg`.
- *Complexity*: Linear in `ranges::distance(rg)`.

[containers.sequences.inplace.vector.capacity] Size and capacity

```
static constexpr size_type capacity() noexcept;
static constexpr size_type max_size() noexcept;
```

- *Returns*: `N`.

```
constexpr void resize(size_type sz);
```

- *Preconditions*: `T` is *Cpp17DefaultInsertable* into `*this`.
- *Effects*: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` default-inserted elements to the sequence.
- *Remark*: If an exception is thrown, there are no effects on `*this`.

```
constexpr void resize(size_type sz, const T& c);
```

- *Preconditions*: `T` is *Cpp17CopyInsertable* into `*this`.
- *Effects*: If `sz < size()`, erases the last `size() - sz` elements from the sequence. Otherwise, appends `sz - size()` copies of `c` to the sequence.
- *Remark*: If an exception is thrown, there are no effects on `*this`.

[containers.sequences.inplace.vector.data] Data

```
constexpr       T* data()       noexcept;
constexpr const T* data() const noexcept;
```

- *Returns*: A pointer such that [`data()` , `data() + size()`) is a valid range. For a non-empty `inplace_vector` , `data() == addressof(front())` .
- *Complexity*: Constant time.

[containers.sequences.inplace.vector.modifiers] Modifiers

```
constexpr iterator insert(const_iterator position, const T& x);
constexpr iterator insert(const_iterator position, T&& x);
constexpr iterator insert(const_iterator position, size_type n, const T& x);
template <class InputIterator>
  constexpr iterator insert(const_iterator position, InputIterator first, InputIte
template <container-compatible-range<T> R>
  constexpr iterator insert_range(const_iterator position, R&& rg);
constexpr iterator insert(const_iterator position, initializer_list<T> il);

template <class... Args> constexpr iterator emplace(const_iterator position, Args&
template<container-compatible-range<T> R>
  constexpr void append_range(R&& rg);
```

- Let *n* be value of `size()` before this call for `append_range` overload, and `distance(begin, position)` otherwise.
- *Complexity*: Linear in the number of elements inserted plus the distance to the end of the vector.
- *Remarks*: If an exception is thrown other than by the copy constructor, move constructor, assignment operator, or move assignment operator of `T` or by any `InputIterator` operation there are no effects. Otherwise, if an exception is thrown, then `size()` ≥ *n* and elements in range `begin()` + [0, *n*) are not modified.

```
constexpr reference push_back(const T& x);
constexpr reference push_back(T&& x);
template <class... Args>
  constexpr reference emplace_back(Args&&... args);
```

- *Returns*: `back()`.
- *Complexity*: Constant.
- *Throws*: `bad_alloc` or any exception thrown by initialization of inserted element.
- *Remarks*: If an exception is thrown there are no effects on `*this`.

```
template <class... Args>
  constexpr pointer try_emplace_back(Args&&... args);
constexpr pointer try_push_back(const T& x);
constexpr pointer try_push_back(T&& x);
```

- Let *vals* denote a pack:
  - `std::forward<Args>(args)...` for the first overload,
  - `x` for the second overload,
  - `set::move(x)` for the third overload.
- *Preconditions*: `T` is *Cpp17EmplaceConstructible* into `*this` from `vals...`.
- *Effects*: If `size() < capacity()` is `true`, appends an object of type `T` direct-non-list-initialized with `vals...`. Otherwise, there are no effects.
- *Returns*: `nullptr` if `size() == capacity()` is `true`, otherwise `addressof(back())`.
- *Complexity*: Constant.
- *Throws*: Nothing unless an exception is thrown by initialization of inserted element.
- *Remarks*: If an exception is thrown there are no effects on `*this`.

```
template <container-compatible-range<T> R>
  constexpr ranges::borrowed_iterator_t<R> try_append_range(R&& rg);
```

- *Preconditions*: `T` is *Cpp17EmplaceConstructible* from `*ranges::begin(rg)` into `*this`.
- *Effects*: Appends copies of initial elements in `rg` before `end()`, until all elements are inserted or `size() == capacity()` is `true`. Each iterator in the range `rg` is dereferenced at most once.
- *Returns*: Iterator past last inserted element of `rg`.
- *Complexity*: Linear in the number of elements inserted.
- *Remarks*: Let *n* be the value of `size()` prior to this call. If an exception is thrown after insertion of *k* elements, then `size()` equals *n* + *k*, elements in the range `begin()` + [0, *n*) are not modified, and elements in the range `begin()` + [*n*, *n* + *k*) correspond to the inserted elements.

```
template <class... Args>
  constexpr reference unchecked_emplace_back(Args&&... x);
```

- *Preconditions*: `size() < capacity()` is `true`.
- *Effects*: Equivalent to: `return *try_emplace_back(std::forward<Args>(args)...);`

```
constexpr reference unchecked_push_back(const T& x);
constexpr reference unchecked_push_back(T&& x);
```

- *Preconditions*: `size() < capacity()` is `true`.
- *Effects*: Equivalent to: `return *try_push_back(std::forward<decltype(x)>(x));`

```
static constexpr void reserve(size_type n);
```

- *Effects*: none.
- *Throws*: `bad_alloc` if `n > capacity()` is `true`.

```
static constexpr void shrink_to_fit() noexcept;
```

- *Effects*: none.

```
constexpr iterator erase(const_iterator position);
constexpr iterator erase(const_iterator first, const_iterator last);
constexpr void pop_back();
```

- *Effects*: Invalidates iterators and references at or after the point of the erase.
- *Throws*: Nothing unless an exception is thrown by the assignment operator or move assignment operator of `T`.
- *Complexity*: The destructor of `T` is called the number of times equal to the number of the elements erased, but the assignment operator of `T` is called the number of times

equal to the number of elements after the erased elements.

[containers.sequences.inplace.vector.erasure] Erasure

```
template<class T, size_t N, class U = T>
  constexpr size_t erase(inplace_vector<T, N>& c, const U& value);
```

- *Effects*: Equivalent to:

```
auto it = remove(c.begin(), c.end(), value);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

```
template<class T, size_t, class Predicate>
  constexpr size_t erase_if(inplace_vector<T, N>& c, Predicate pred);
```

- *Effects*: Equivalent to:

```
auto it = remove_if(c.begin(), c.end(), pred);
auto r = distance(it, c.end());
c.erase(it, c.end());
return r;
```

[version.syn] (http://eel.is/c++draft/version.syn)

Add:

```
#define __cpp_lib_inplace_vector   20XXXXL // also in <inplace_vector>
```

[diff.cpp23.library] (https://eel.is/c++draft/diff.cpp23.library) Compatibility

Modify:

- 2 (https://eel.is/c++draft/diff.cpp23.library#2) Affected subclause: [headers] (https://eel.is/c++draft/headers)
- Change: New headers.
- Rationale: New functionality.
- Effect on original feature: The following C++ headers are new: <debugging>, <hazard_pointer>, `<inplace_vector>`, <linalg>, <rcu>, and <text_encoding>. Valid C++ 2023 code that #includes headers with these names may be invalid in this revision of C++.

Acknowledgments

This proposal is based on Boost.Container's `boost::container::static_vector`, mainly authored by Adam Wulkiewicz, Andrew Hundt, and Ion Gaztanaga. The reference implementation is based on Howard Hinnant `std::vector` implementation in libc++ and its test-suite. The following people provided valuable feedback that influenced some aspects of this proposal: Walter Brown, Zach Laine, Rein Halbersma, Andrzej Krzemieński, Casey Carter, Tomasz Kamiński, and many others. Many thanks to Daniel Krügler for reviewing the wording.